

Assignment 1 - A* Walk aware directions

Track B implementation

Ali Abbas (24968341)

1.0 What I built

What I designed here is a walk aware A* implementation with the usage of real world data collected from OpenStreetMap's [Overpass API](#) & Sydney NSWs GTFS feed. The idea of it is based on the standard implementation of A* but with additional constraints that allow the user to specify how much walking they would like to include in their route.

Instead of only finding the fastest route from point A to point B, the algorithm also considers the amount of walking along the route. This means that reaching the same location through two different paths may produce two different search states if the amount of walking completed is different. A slightly slower route may therefore still be useful if it better satisfies the user's walking requirement.

This was made in GoLang as this is one of the languages I'm strong at using, there was no need for additional libraries as everything that is made in this is either within GoLang standard library or is primarily an API call to a RESTful api to collect data.

Before running the code it is important that you register on <https://opendata.transport.nsw.gov.au/> and make a key to pull data about GTFS so you get transport data so then you can use this app, set this as an env variable and run the importer, once the importer is ran you can then run the server to test this application.

2.0 The tool & what it does

The tool that was built has two main parts, an importer which pulls data and builds a navigation network from OpenStreetMap and a server that loads this network and serves it through a RESTful API (and has a web view but this is for demonstration purposes to see it work the reality is this would just be a RESTful API called to calculate routes)

2.1 Building the Navigation Graph

The importer collects data from OpenStreetMap using the Overpass API. The Overpass API is special because it allows us to limit the requested data to a specified bounding box (say from Sydney CBD to Glenfield as an example). This is vital for testing this as we do not want to traverse EVERYTHING we want a limited sample space so we can test if this works effectively.

The returned OpenStreetMap data is then converted into a graph. OpenStreetMap node IDs are kept as the graph node IDs, while pathways are broken into edges between consecutive nodes. Due to the fact the nodes are just latitude and longitude the AI suggested using the Haversine

formula which is used to calculate the distance between two locations and then we convert that to walking time based on a configurable walking speed defined.

The graph itself is stored as an adjacency list:

```
type Graph struct {
    Nodes map[model.NodeID]model.Node
    Edges map[model.NodeID][]model.Edge
}
```

This allows the routing algorithm to efficiently access all edges leaving the current node in $O(1)$ time complexity.

2.2 Adding Public Transport

Public transport data can be imported from a [GTFS](#) feed.

The importer reads the GTFS stops.txt, routes.txt, trips.txt and stop_times.txt files. These are converted into bus and train edges between transport stops.

Each transport stop is then connected to its nearest OpenStreetMap walking node, provided it is within the configured maximum transfer distance. This combines the pedestrian and transport data into one graph that can contain:

```
walk -> train -> walk
walk -> bus -> walk
walk -> bus -> train -> walk
```

The resulting edge structure contains the transport mode, duration, distance and optional service name.

2.3 Walk Aware A*

The main modification that was made to A* is that the search state cannot simply contain the current location.

The state contains:

```
type StateKey struct {
    Node      model.NodeID
    WalkingUnits int64
}
```

The important addition is WalkingUnits. This allows the algorithm to distinguish between two routes that arrive at the same location with different amounts of walking already completed.

Walking progress is stored as integer units and capped once the requested walking minimum has been reached. This prevents the number of possible states from continuing to increase after the walking goal has already been satisfied.

For example, reaching the same node with:

Route A 12 minutes total 3 minutes walking	Route B 14 minutes total 10 minutes walking
---	--

cannot always be treated as the same state. If the user asked for at least 15 minutes of walking, Route B may be more useful even though it took longer to reach that node.

2.4 Examples

PLAN A TRIP

Where are you going?

FROM

University Of Technology Sydney, E

TO

Glenfield Station

MINIMUM WALKING

0 min10 minutes30 min

The selected route must include at least this much walking.

Find route ->

YOUR ROUTE

To Glenfield Station

41.310.1846.611787411

total minuteswalking minuteswalking metrestransfersA* states explored

Fastest route with at least 10 minutes walking

01 > Walk to Central Station, Platform 25
7.9 min / 659.5 m

02 [T] Take T4 to Wolli Creek Station, Platform 4
8.7 min / 6,826.5 m

03 > Transfer at Wolli Creek Station
1.4 min / 118.5 m

04 [T] Take T8 to Glenfield Station, Platform 4
22.5 min / 24,746.5 m

05 > Transfer at Glenfield Station
0.8 min / 68.6 m

PLAN A TRIP

Where are you going?

FROM

University Of Technology Sydney, f

TO

Glenfield Station

MINIMUM WALKING

0 min

20 minutes

30 min

The selected route must include at least this much walking.

Find route ->

YOUR ROUTE

To Glenfield Station

60.1

27.1

2,278.9

2

1712104

total minutes

walking minutes

walking metres

transfers

A* states explored

Fastest route with at least 20 minutes walking

01

>

Walk to Town Hall Station, Platform 4

24.9 min / 2,091.8 m

02

[T]

Take SCO to Redfern Station, Platform 12

3.4 min / 2,345.6 m

03

[T]

Take T4 to Wolli Creek Station, Platform 4

7.1 min / 5,783.9 m

04

>

Transfer at Wolli Creek Station

1.4 min / 118.5 m

05

[T]

Take T8 to Glenfield Station, Platform 4

22.5 min / 24,746.5 m

06

>

Transfer at Glenfield Station

0.8 min / 68.6 m

The examples above show how increasing the minimum walking requirement changes the resulting route. With a minimum of 10 minutes walking the route takes 41.3 minutes and includes 10.1 minutes of walking. Increasing this requirement to 20 minutes causes the algorithm to take a different route which takes longer overall but includes 27.1 minutes of walking. This shows the walking constraint actually affects the search rather than just being calculated after the route is found.

3.0 What I learnt

One of the main things I learned from this project was that modifying a shortest path algorithm with an additional constraint changes the way the search state has to be represented.

Initially, it seemed like reaching the same node twice would mean that only the fastest route to that node needed to be kept. However, this is not always true when a minimum walking requirement is involved. A route that reaches a node slightly slower may have accumulated more walking and may therefore be more useful later in the search. This led to the state being represented using both the current node and the walking progress rather than only the node itself.

Another thing I learned was how real world navigation data differs from the clean graphs normally used when learning graph algorithms. The OpenStreetMap data first had to be filtered and converted into nodes and edges, while the GTFS transport data had to be connected to nearby walking nodes before it could be used by the routing algorithm.

I also learned that route cost is not necessarily just one value. My implementation keeps track of travel time, walking progress and transfers, and these can be compared differently depending on whether the user prefers the fastest route, the least walking or the fewest transfers.

Overall, the main thing I learned was that applying an algorithm such as A* a real problem involves much more than implementing the basic algorithm. The representation of the graph, the definition of a state, the heuristic and the constraints all directly affect whether the result is correct and useful.

4.0 AI usage

I used ChatGPT throughout the project quite heavily for planning, discussing algorithm design decisions, reviewing code and helping debug parts of the implementation. This ended up being a fairly large project with a lot of different moving parts, including the OpenStreetMap importer, GTFS parsing and merging, graph construction, the routing algorithm, API, server and frontend. Because of the amount of work involved and the time available, I relied on AI a lot to help scaffold and iterate on the implementation so I could get all of these parts working together in time.

One of the main areas where AI was useful was helping me reason about how the A* state needed to change once a walking requirement was introduced. The initial idea of storing only the current node was not sufficient because two routes reaching the same location could have different amounts of walking completed. This eventually led to the state containing additional information such as walking progress, the previous transport service and whether the user was currently on public transport.

I also used AI to scaffold the initial implementation of the project. At first I incorrectly explained that I wanted a maximum walking constraint when what I actually wanted was a minimum walking goal. This resulted in the first version ignoring routes once too much walking had been done, which was effectively the opposite of the behaviour I wanted. After testing this with mock data, I worked with the Codex to fit the requirement and the search logic was redesigned around walking time until the minimum requirement has been satisfied.

I then used ChatGPT Codex to continue iterating on the project, particularly for larger implementation tasks such as the GTFS integration, frontend, API wiring and fixing issues that appeared once real data was introduced. Some specific issues that came up were:

1. Maximum vs minimum walking: I initially gave the AI the wrong requirement, which caused the route logic to work in the opposite direction to what I intended. I fixed this after testing and changed the search state and goal condition to work around a minimum walking target instead.
2. Very short public transport edges: During testing, the algorithm heavily preferred public transport and would sometimes take a bus for around one minute when continuing to walk would have made more sense. I changed the cost model to include a boarding penalty so that very short public transport trips were less attractive.
3. Overpass API timeouts: Fetching data for larger bounding boxes caused repeated timeouts. Instead of just continuing to increase the timeout, I changed the importer so

the requested area could be split into smaller overlapping tiles, downloaded separately and then merged together.

I understand the A* search itself, why walking progress has to be part of the state, how the heuristic is calculated and how the OSM and GTFS data is represented as a graph. I relied more heavily on AI for scaffolding some of the larger implementation pieces, especially GTFS parsing, frontend code and parts of the importer, mainly because the scope of the project was larger than just implementing the algorithm itself. I still tested these parts and made changes when the behaviour did not match what I expected.

Overall, AI was a major part of how I was able to complete the project within the available time. It helped reduce the amount of time spent writing boilerplate and connecting the different components together, while I focused more on testing the behaviour, understanding the algorithmic changes and fixing the cases where the generated implementation did not match the intended design.